Strings - indexing, slicing, and methods

Today

Create string variables and investigate characters, length, and membership.

Extract characters and substrings using indexing and slicing.
Use string methods and explain why the original string stays unchanged.

TERMINOLOGY

string

An ordered sequence of characters representing text.

Code

```
1 print("Hello World")
```

Printed output

TERMINOLOGY

string

An ordered sequence of characters representing text.

Code

```
1 print("Hello World")
```

Printed output

```
Hello World
```

character

One element of a string, including a letter, digit, punctuation mark, or space.

Code

```
1 print("R")
```

Printed output

character

One element of a string, including a letter, digit, punctuation mark, or space.

Code

```
1 print("R")
```

Printed output

```
R
```

TERMINOLOGY

string literal

Text written directly in source code inside quotation marks.

Code

```
1 name = "Ryan"
2 print(name)
```

Printed output

string literal

Text written directly in source code inside quotation marks.

Code

```
1 name = "Ryan"
2 print(name)
```

Printed output

```
Ryan
```

CODE EXAMPLE

Strings and your name

A **string** is an ordered sequence of characters. Letters, punctuation, and spaces are characters. Quotes mark string literals.

Code

```
1 firstName = "Ryan"
2 lastName = "McDaniel"
3 print(firstName, lastName)
```

Printed output

CODE EXAMPLE

Strings and your name

A **string** is an ordered sequence of characters. Letters, punctuation, and spaces are characters. Quotes mark string literals.

Code

```
1 firstName = "Ryan"
2 lastName = "McDaniel"
3 print(firstName, lastName)
```

Printed output

```
Ryan McDaniel
```

EXERCISE

Exercise 0 — Strings and your name

Create `firstName` and `lastName` using your name or an invented name, then `print` them. Single or double quotes both work.

```
1 firstName = ""
2 lastName = ""
3 # YOUR CODE HERE: fill in the names, then print them.
```

index

An integer position used to select a character. The first index is `0`.

Code

```
1 print("Something"[0])
```

Printed output

index

An integer position used to select a character. The first index is `0`.

Code

```
1 print("Something"[0])
```

Printed output

```
S
```

CODE EXAMPLE

Indexing selects one character

An **index** is a character's position. Python starts at **0**. Square brackets select the character. The gray code line numbers are separate from character indices.

Code

```
1 word = "Something"
2 #       012345678
3 print(word[0])
4 print(word[3])
```

Printed output

CODE EXAMPLE

Indexing selects one character

An **index** is a character's position. Python starts at **0**. Square brackets select the character. The gray code line numbers are separate from character indices.

Code

```
1 word = "Something"
2 #       012345678
3 print(word[0])
4 print(word[3])
```

Printed output

```
S
e
```

length

The number of characters in a string, including spaces.

Code

```
1 print(len("MyString"))
```

Printed output

length

The number of characters in a string, including spaces.

Code

```
1 print(len("MyString"))
```

Printed output

```
8
```

membership

Whether text occurs within another string. **`in`** returns **`True`** or **`False`**.

Code

```
1 print("s" in "string")
```

Printed output

membership

Whether text occurs within another string. **`in`** returns **`True`** or **`False`**.

Code

```
1 print("s" in "string")
```

Printed output

```
True
```

keyword

A reserved word in Python syntax, such as **`in`**. It cannot be a variable name.

Code

```
1 print("s" in "string")
```

Printed output

keyword

A reserved word in Python syntax, such as **`in`**. It cannot be a variable name.

Code

```
1 print("s" in "string")
```

Printed output

```
True
```

CODE EXAMPLE

String facts: length and membership

`len` counts characters. The `in` keyword tests whether text occurs within a string. Uppercase and lowercase differ.

Code

```
1 print(len("MyString"))
2 print("s" in "string")
3 print("z" in "string")
```

Printed output

CODE EXAMPLE

String facts: length and membership

`len` counts characters. The `in` keyword tests whether text occurs within a string. Uppercase and lowercase differ.

Code

```
1 print(len("MyString"))
2 print("s" in "string")
3 print("z" in "string")
```

Printed output

```
8
True
False
```

EXERCISE

Exercise 1 — String facts: length and membership

Print the length of `firstName`. Test whether uppercase `"R"` occurs in `lastName`. Try lowercase `"r"` too. Predict each result before running.

```
1 # YOUR CODE HERE: length and membership tests on the names.
```

EXERCISE

Exercise 2 — Guess the word

Your instructor has a hidden word. Choose questions that help the class discover it.

```
1 len(secretString)
2 'a' in secretString
3 secretString[0]
```

What does each answer tell you? Which question will you ask first?

substring

A contiguous sequence of characters within a string.

Code

```
1 print("and" in "Sand")
```

Printed output

substring

A contiguous sequence of characters within a string.

Code

```
1 print("and" in "Sand")
```

Printed output

```
True
```

TERMINOLOGY

concatenation

Combining strings into a new string using `+`.

Code

```
1 print("Some" + "thing")
```

Printed output

concatenation

Combining strings into a new string using `+`.

Code

```
1 print("Some" + "thing")
```

Printed output

```
Something
```

CODE EXAMPLE

Substrings using individual indices

A **substring** is a contiguous part of a string: `"and"` is within `"Sand"`.

How can we build `"Some"` from `"Something"` using individual indices?

Code

```
1 something = "Something"
2 print(something[0])
```

Printed output

CODE EXAMPLE

Substrings using individual indices

A **substring** is a contiguous part of a string: `"and"` is within `"Sand"`.

How can we build `"Some"` from `"Something"` using individual indices?

Code

```
1 something = "Something"
2 print(something[0])
```

Printed output

```
S
```

EXERCISE

Exercise 3 — Substrings using individual indices

Use individual indices and `+` to `print "Some"`. Do this once before trying the shorter slice form on the next slide.

```
1 something = "Something"
2 # YOUR CODE HERE: combine the first four characters.
```

slice

A selection using `[start:stop]`. Start is included and stop is excluded.

Code

```
1 print("Something"[0:4])
```

Printed output

slice

A selection using `[start:stop]`. Start is included and stop is excluded.

Code

```
1 print("Something"[0:4])
```

Printed output

```
Some
```

expression

Code that evaluates to a value, such as `len(firstName)` or `something[0:4]`.

Code

```
1 print(len("Something") - 1)
```

Printed output

expression

Code that evaluates to a value, such as `len(firstName)` or `something[0:4]`.

Code

```
1 print(len("Something") - 1)
```

Printed output

```
8
```

CODE EXAMPLE

Slicing selects a substring

A **slice** uses `string[start:stop]`. Include the start. Exclude the stop.

Code

```
1 something = "Something"
2 #            012345678
3 print(something[0:4])
4 print(something[4:9])
```

Printed output

CODE EXAMPLE

Slicing selects a substring

A **slice** uses `string[start:stop]`. Include the start. Exclude the stop.

Code

```
1 something = "Something"
2 #            012345678
3 print(something[0:4])
4 print(something[4:9])
```

Printed output

```
Some
thing
```

Exercise 4 — Slicing selects a substring

Print `"Some"` using a slice. Then `print "thing"` with another slice and explain your start and stop.

```
1 something = "Something"
2 # YOUR CODE HERE: print two slices.
```

method

A function accessed through a value using a dot, such as `firstName.upper()`.

Code

```
1 print("NoIce".lower())
```

Printed output

method

A function accessed through a value using a dot, such as `firstName.upper()`.

Code

```
1 print("NoIce".lower())
```

Printed output

```
noice
```

TERMINOLOGY

chaining

Applying another method to the result of a method call.

Code

```
1 print("NoIce".lower().capitalize())
```

Printed output

chaining

Applying another method to the result of a method call.

Code

```
1 print("NoIce".lower().capitalize())
```

Printed output

```
Noice
```

immutable

Cannot be changed in place. String methods return new strings.

Code

```
1 original = "NoIce"
2 changed = original.lower()
3 print(original, changed)
```

Printed output

immutable

Cannot be changed in place. String methods return new strings.

Code

```
1 original = "NoIce"
2 changed = original.lower()
3 print(original, changed)
```

Printed output

```
NoIce noice
```

Methods, chaining, and immutability

Predict each result. Methods return new strings. The original stays unchanged.

Code

```
1 noIce = "NoIce"
2 result = noIce.lower().capitalize()
3 print(result)
4 print(noIce)
```

Printed output

Methods, chaining, and immutability

Predict each result. Methods return new strings. The original stays unchanged.

Code

```
1 noIce = "NoIce"
2 result = noIce.lower().capitalize()
3 print(result)
4 print(noIce)
```

Printed output

```
Noice
NoIce
```

EXERCISE

Exercise 5 — Methods, chaining, and immutability

Print capitalized, uppercase, and lowercase versions of `firstName`. Then transform `"NoIce"` into `"Noice"` in one line. Print the original name afterward.

```
1 noIce = "NoIce"
2 # YOUR CODE HERE: try case methods on firstName and noIce.
```

negative index

A position counted from the end of a string. `-1` selects its last character.

Code

```
1 print("Something"[-1])
```

Printed output

negative index

A position counted from the end of a string. `-1` selects its last character.

Code

```
1 print("Something"[-1])
```

Printed output

```
g
```

CODE EXAMPLE

Practice reference: omitted bounds and negative indices

Omit a bound to use that end of the string. Negative indices count from the end. `-1` is the last character.

Code

```
1 something = "Something"
2 print(something[:4])
3 print(something[4:])
4 print(something[-5:])
```

Printed output

CODE EXAMPLE

Practice reference: omitted bounds and negative indices

Omit a bound to use that end of the string. Negative indices count from the end. `-1` is the last character.

Code

```
1 something = "Something"
2 print(something[:4])
3 print(something[4:])
4 print(something[-5:])
```

Printed output

```
Some
thing
thing
```

TERMINOLOGY

delimiter

Text used to separate pieces when splitting or joining a string.

Code

```
1 print("No ice".split(" "))
```

Printed output

delimiter

Text used to separate pieces when splitting or joining a string.

Code

```
1 print("No ice".split(" "))
```

Printed output

```
['No', 'ice']
```

list

An ordered collection of values. `split` returns a list of string pieces.

Code

```
1 print("No ice".split(" "))
```

Printed output

list

An ordered collection of values. `split` returns a list of string pieces.

Code

```
1 print("No ice".split(" "))
```

Printed output

```
['No', 'ice']
```

CODE EXAMPLE

Practice reference: split, join, and delimiter

A **delimiter** separates pieces. `split` returns a list of strings. `join` combines pieces with the delimiter between them.

Code

```
1 text = "Make like a banana and split!"
2 pieces = text.split("banana")
3 print(pieces)

4 print("banana".join(pieces))
```

Printed output

CODE EXAMPLE

Practice reference: split, join, and delimiter

A **delimiter** separates pieces. `split` returns a list of strings. `join` combines pieces with the delimiter between them.

Code

```
1 text = "Make like a banana and split!"
2 pieces = text.split("banana")
3 print(pieces)

4 print("banana".join(pieces))
```

Printed output

```
['Make like a ', ' and split!']
Make like a banana and split!
```

Practice reference: replace

`replace(search, replacement)` returns text with matching substrings replaced. Search text is case-sensitive.

Code

```
1 text = "Make like a banana and split!"
2 print(text.replace("banana", "pear"))

3 print(text)
```

Printed output

CODE EXAMPLE

Practice reference: replace

`replace(search, replacement)` returns text with matching substrings replaced. Search text is case-sensitive.

Code

```
1 text = "Make like a banana and split!"
2 print(text.replace("banana", "pear"))

3 print(text)
```

Printed output

```
Make like a pear and split!
Make like a banana and split!
```

Application: finding text in documents

Document-processing tools search for phrases and extract nearby text.

Code

```
1 text = "Student: Ryan"
2 name = text[9:]
3 print(name)
```

Printed output

Which substring is extracted? How would a different prefix affect this slice?

Application: finding text in documents

Document-processing tools search for phrases and extract nearby text.

Code

```
1 text = "Student: Ryan"
2 name = text[9:]
3 print(name)
```

Printed output

```
Ryan
```

Which substring is extracted? How would a different prefix affect this slice?

Exercise 6 — Negative slice indices

Use negative slice indices to `print` `"Some"` from `"Something"`. Then extract `"thing"` with a negative start. Explain how the positions relate to the string's length.

```
something = "Something"
# YOUR CODE HERE: select Some and thing with negative bounds.
```

Optional extension

Predict `something[:4]`, `something[0:-1]`, and `something[-1:-4]`. Check them and explain why the last expression does not reverse the string.

Exercise 7 — Split, join, and replace

Use `split` and `join` to transform `"No ice"` into `"Noice"`. Then use `replace` to transform `"Not Ice"` into `"Notice"`, and that result into `"Noice"`.

```
spaced = "No ice"
phrase = "Not Ice"
# YOUR CODE HERE: create and print the requested strings.
```

Exercise 8 — The mystery phrase

The original challenge puts `"N O I C E"` after your names. Verify that the phrase is present, extract it as a substring, and find a second way to extract it. Share your reasoning with a neighbor.

```
1 mysteryString = firstName + lastName + "N O I C E"
2 # YOUR CODE HERE: membership test and two extractions.
```

Exercise 9 — Two ways to transform the phrase

Find two ways to `replace` every space in `"N O I C E"` with a comma. Then transform the original phrase into `"noice"`.

```
1 phrase = "N O I C E"
2 # YOUR CODE HERE: two comma versions, then lowercase noice.
```

Optional extension

How many ways can you combine `firstName` and `lastName` with a single space between them? Try concatenation and joining pieces. Distinguish building one string from printing multiple arguments.